ĐỒ ÁN 1

THIẾT KẾ PHẦN CỨNG TĂNG TỐC AUTOENCODER KHỬ NHIỄU ẢNH TRÊN FPGA

Giảng viên hướng dẫn: TS. Phạm Hoài Luân

Sinh viên thực hiện:

Bùi Tấn Đạt - 23520244
Lương Ngọc Đại - 23520215

TỔNG QUAN HƯỚNG TIẾP CẬN

Đồ án xây dựng pipeline hai giai đoạn chạy trên FPGA (KV260):
[Ảnh nhiễu] -> Denoiser (Autoencoder) -> [Ảnh sạch] -> Classifier -> [Số 0–9]

Giai đoạn 1 - Denoising Autoencoder:
Loại bỏ nhiễu Gaussian khỏi ảnh chữ số MNIST 28×28 bằng mạng convolutional autoencoder. Đây là phần trọng tâm của đồ án.

Giai đoạn 2 - Classifier:
Nhận ảnh đã khử nhiễu, nhận dạng chữ số 0–9 bằng mạng LeNet nhẹ.

BƯỚC 1 — PYTHON: THUẬT TOÁN DENOISING

Mô tả thuật toán:

Thuật toán được sử dụng là Convolutional Denoising Autoencoder (CDAE)
Nguyên lý hoạt động của thuật toán:
- Encoder học cách nén đặc trưng quan trọng của ảnh, đồng thời lọc bỏ nhiễu ngẫu nhiên.
- Decoder tái tạo lại ảnh sạch từ biểu diễn đã nén.
- Mạng được huấn luyện với cặp (ảnh nhiễu → ảnh sạch), hàm mất mát MSE buộc output càng gần ảnh gốc càng tốt.

Loại nhiễu: Nhiễu Gaussian ($\sigma = 0.5$), cộng vào pixel sau đó clamp về [0, 1].

KIẾN TRÚC DENOISING AUTOENCODER (665 tham số)

Input: 28×28×1 [MINST chữ số]
Phần Encoder:

Conv2d(1→4, kernel=3, padding=1) + ReLU → 28×28×4
MaxPool2d(2) → 14×14×4
Conv2d(4→8, kernel=3, padding=1) + ReLU → 14×14×8

Phần Decoder:

Upsample(×2, nearest neighbor) → 28×28×8
Conv2d(8→4, kernel=3, padding=1) + ReLU → 28×28×4
Conv2d(4→1, kernel=3, padding=1) + Sigmoid → 28×28×1

Output: 28×28×1 (ảnh đã khử nhiễu, giá trị trong [0, 1])

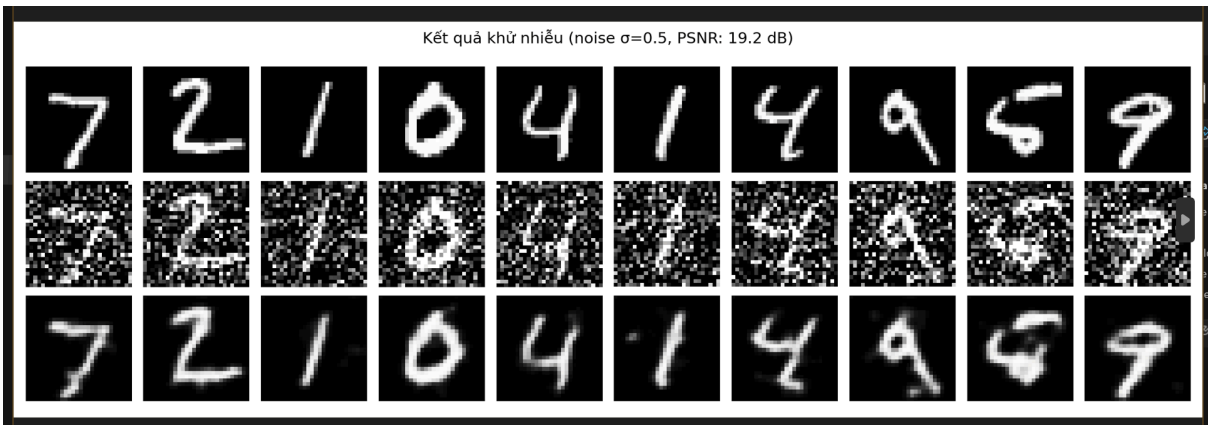
Lý do chọn kiến trúc nhỏ (665 params):
- Phù hợp tài nguyên hạn chế của FPGA (BRAM, DSP, LUT).
- Đủ năng lực xử lý nhiễu Gaussian mức độ vừa (sigma = 0.5).
- Dễ chuyển đổi sang C++/HLS ở các bước tiếp theo

Quá trình train:

Thiết lậ:
Framework : PyTorch
Dataset : MNIST (60,000 train / 10,000 test), ảnh xám 28×28
Hàm mất mát: MSE Loss (Mean Squared Error)
Optimizer : Adam, learning rate = 0.001
Batch size : 128 (Cân bằng tốt giữa tốc độ và chất lượng)
Epochs : 50 (mô hình đã học qua toàn bộ tập dữ liệu 50 lần)

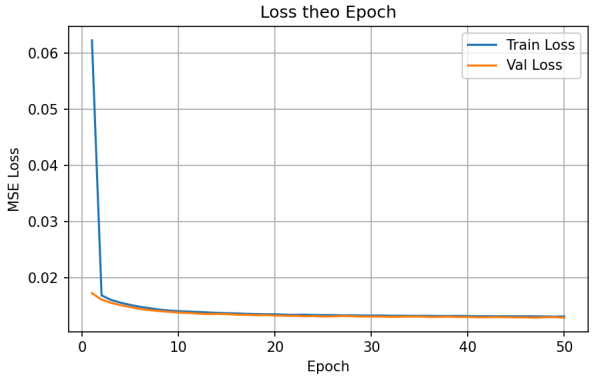
Kết quả train:
Đánh giá trên 10,000 ảnh test:
File đầu ra:
comparison.png - ảnh 3 hàng: gốc / nhiễu / khử nhiễu
training_curves.png - biểu đồ Loss & PSNR theo epoch
metrics.json - số liệu PSNR, MSE
denoiser.pth - weights model
Kết quả ở bước như ảnh

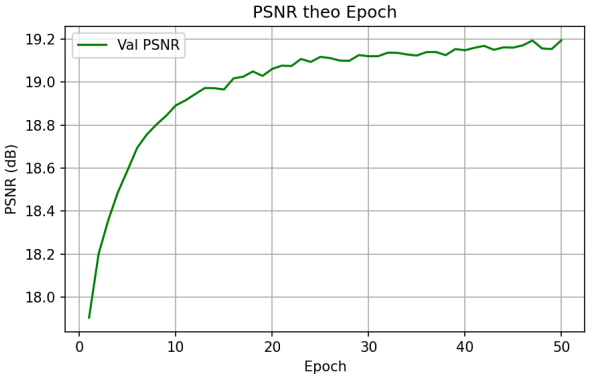
Name	Date modified	Type	Size
comparison.png	28/03/2026 11:13 pm	PNG File	65 KB
denoiser.pth	28/03/2026 11:13 pm	PTH File	7 KB
history.json	28/03/2026 11:13 pm	JSON Source File	5 KB
metrics.json	28/03/2026 11:13 pm	JSON Source File	1 KB
training_curves.png	28/03/2026 11:13 pm	PNG File	62 KB



comparasion.png

Với noise sigma = 0.5, ta thấy ảnh nhiễu trắng rất khó nhìn và nhiễu nặng.
Ảnh nhiễu được đánh giá so với ảnh gốc là PSNR = 9.38 dB
Thu được kết quả ảnh với PSNR = 19.187 dB cải thiện 9.9 dB





Nhận xét:

- PSNR cải thiện +9.81 dB chứng minh thuật toán denoising hoạt động hiệu quả.
- MSE giảm ~9 lần so với ảnh nhiễu ban đầu.
- Train loss và val loss gần bằng nhau → mô hình không overfit


```
{
  "noise_std": 0.5,
  "epochs": 50,
  "learning_rate": 0.001,
  "total_params": 665,
  "final_train_loss": 0.013098106421033541,
  "final_val_loss": 0.012868234537541866,
  "mse_noisy": 0.11563813394006295,
  "mse_denoised": 0.012869432580339003,
  "psnr_noisy_db": 9.37922410119938,
  "psnr_denoised_db": 19.18731129320362,
  "psnr_improvement_db": 9.80808719200424,
  "timestamp": "2026-03-28 23:13:13"
}
```

Bước 2 - HIỆU QUẢ THUẬT TOÁN DENOISING

Dùng một classifier độc lập để định lượng mức độ cải thiện chất lượng ảnh sau khi qua thuật toán denoising. Nếu ảnh khử nhiễu tốt hơn, classifier sẽ nhận dạng chính xác hơn so với ảnh nhiễu.

KIẾN TRÚC CLASSIFIER (37,106 tham số — LeNet nhẹ)

```
Input: 28×28×1

Conv2d(1→6, kernel=5, padding=2) + ReLU -> 28×28×6
MaxPool2d(2)  -> 14×14×6
Conv2d(6→16, kernel=5) + ReLU -> 10×10×16
MaxPool2d(2)  -> 5×5×16
Flatten ->  400
Linear(400->84) + ReLU  -> 84
Linear(84->10)                -> 10 (logits)

Output: xác suất cho 10 chữ số (0–9)
```

Classifier được huấn luyện trên ảnh SẠCH, sau đó đánh giá trên 3 kịch bản để đo ảnh hưởng của nhiễu và hiệu quả của denoise

Thiết lập train:

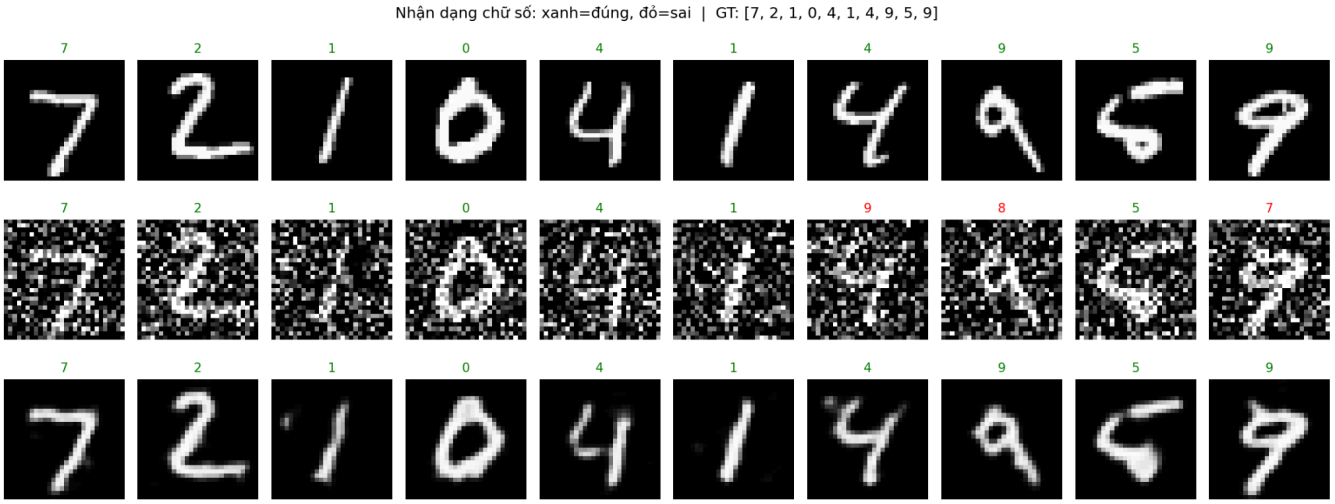
- Framework : PyTorch
- Hàm mất mát: CrossEntropyLoss
- Optimizer : Adam, lr = 0.001
- Batch size : 128
- Epochs : 15 (đủ để hội tụ)

Đánh giá trên 10,000 ảnh test:

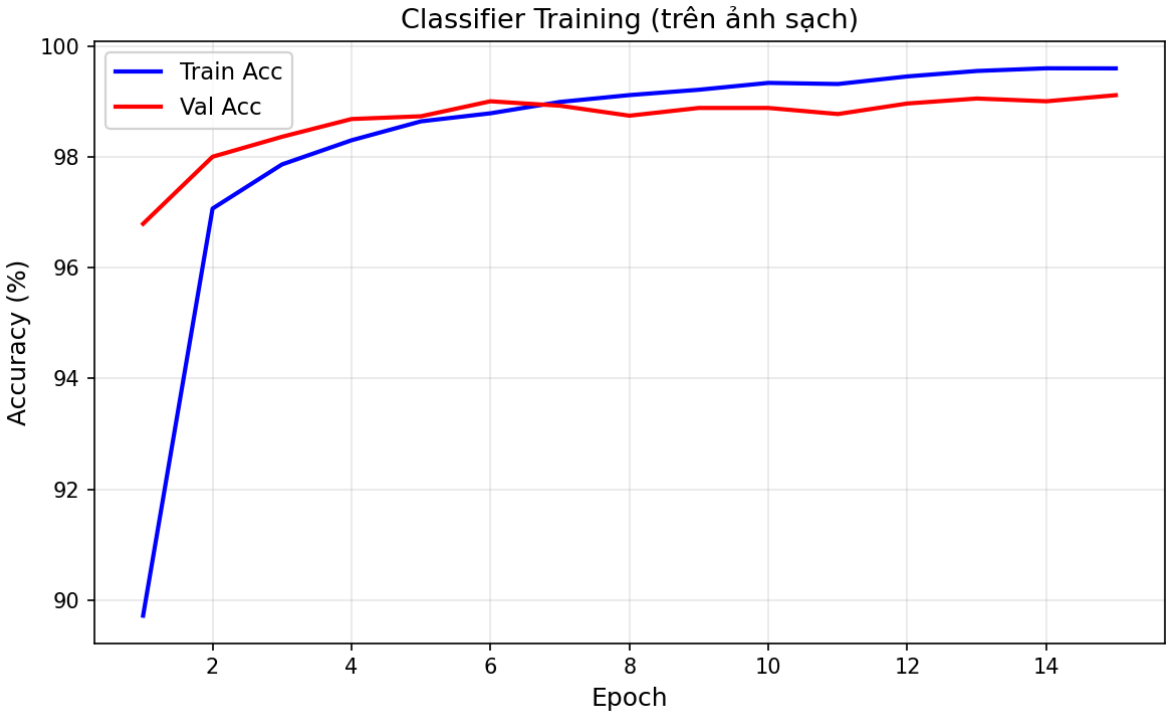
Kịch bản	Độ chính xác
Ảnh sạch (baseline)	99.11%
Ảnh nhiễu gaussian với sigma = 0.5	88.90%
Ảnh qua thuật toán Denoising	96.30%

Nhận thấy:

- Cải thiện sau denoising: +7.40%
- Phục hồi so với baseline: ~84.7%



Dự đoán ảnh và so sánh giữa 3 kịch bản.



Train acc: accuracy trên tập train (60,000 ảnh)
Val acc: accuracy trên tập test (10,000 ảnh)
Nhận thấy:
Tại Epoch 1 thì Val acc cao hơn rất nhiều so với Train acc do Model chưa học đủ.
Từ Epoch 2–5 thì Train Acc tăng nhanh, bắt kịp Val Acc
Từ Epoch 6 - 15 thì Cả 2 hội tụ ổn định ~99%
=> Biểu đồ này cho thấy classifier train ổn định và tốt.
Nhận xét:

- Nhiễu Gaussian sigma =0.5 làm giảm accuracy nghiêm trọng: 99.11% xuống còn 88.90% (giảm 10.21%).
- Sau khi qua thuật toán denoising: accuracy phục hồi lên 96.30%,

tức cải thiện +7.40% so với ảnh nhiễu.
- Thuật toán denoising phục hồi được ~84.7% lượng accuracy bị mất do nhiễu - chứng minh hiệu quả rõ ràng.

Kết luận 2 bước đầu

Thuật toán Convolutional Denoising Autoencoder (665 tham số) đã được huấn luyện thành công và cho thấy hiệu quả rõ ràng:

- PSNR cải thiện +9.81 dB trên tập MNIST test
- Accuracy nhận dạng chữ số phục hồi từ 88.9% lên 96.3% sau khi qua denoiser (+7.4%)

Mô hình nhỏ gọn (665 params) phù hợp để chuyển sang C++/HLS và triển khai trên FPGA KV260 ở các bước tiếp theo.

Các bước tiếp theo

Bước 3: Xuất weights từ PyTorch (.pth) ra mảng C header (.h)
Bước 4: Viết C++ inference cho pipeline denoiser → classifier
Bước 5: Quantization fixed-point (ap_fixed trong Vitis HLS)
Bước 6: Tổng hợp HLS → IP Core (C-sim, Cosim, Synthesis)
Bước 7: Tích hợp SoC trên KV260, demo end-to-end

Tham khảo

Các bài báo liên quan trực tiếp đến thuật toán Denoising Autoencoder được sử dụng trong đồ án:

[1] P. Vincent, H. Larochelle, Y. Bengio, P.-A. Manzagol.
"Extracting and Composing Robust Features with Denoising Autoencoders."
ICML 2008.
Bài báo gốc đề xuất thuật toán Denoising Autoencoder, nền tảng lý thuyết cho phương pháp huấn luyện cặp (ảnh nhiễu → ảnh sạch) được áp dụng trong đồ án.

[2] D. Mascharka, R. Manley.
"A Deep Convolutional Auto-Encoder with Pooling-Unpooling Layers in Caffe."
arXiv:1701.04949, 2017.
Kiến trúc Convolutional Autoencoder trên ảnh MNIST, tham khảo trực tiếp cho thiết kế encoder-decoder (Conv → Pool → Upsample) trong đồ án.

[3] A. Soltani et al.
"An Energy-Efficient Reconfigurable Autoencoder Implementation on FPGA."
arXiv:2301.07050, 2023.
Tham khảo triển khai kiến trúc Autoencoder lên FPGA, đánh giá tài nguyên (BRAM, DSP, LUT) và hiệu năng năng lượng cho các bước HLS và SoC tiếp theo.